Spout SETTLEMENT WORKER

# Every order, *accounted for.*

Manual end-to-end run on the hosted stack, 7 September 2026. Seven steps driven from the operator console: a normal order, a broker outage, a market closure, a kill switch, a reconciliation and a triple replay. Every observation in this report is copied from the order ledgers and can be verified on Solana devnet.

| | |
|---|---|
| STACK | NestJS · TypeORM · PostgreSQL · Solana devnet · Alpaca sandbox (mock during this run) |
| API | https://api-production-1d80.up.railway.app |
| CONSOLE | https://spout-settlement-console.vercel.app |
| WALLET | BLcWH9MWxKkFNNZj7mBm3vF7aiZieG3BnZMWXtoiktoC |
| PROGRAM | DB2h5equ9Qp2qaaeKyL9sL6RD8LAhZzjiSUYgnaW3aeF |
| SESSION | 2026-09-07 · 13:47 to 14:27 UTC |

01 · SUMMARY

# Seven steps, one invariant: one trade, one settlement.

The pipeline turns a devnet `BuyOrderCreated` into exactly one broker trade and exactly one on-chain settlement, whatever fails in between. Each step below injected one failure, or none, and the ledger recorded what happened.

ORDERS PLACED

4

3 settled, 1 refunded

DUPLICATE TRADES OR SETTLEMENTS

0

also 0 alerts, breaker not tripped

| STEP | ACTION | EXPECTED | OBSERVED |
|---|---|---|---|
| 1 | Save the admin token, resume the workers | `GET /workers` answers `paused: false` | `confirmed 13:47` |
| 2 | Place an order (AAPL, 5 USDC) from the console | `SETTLED` in seconds, one broker order, one settlement | `settled in 7 s, no retries` |
| 3 | Simulator: HTTP 500 for 30 s, then an order | Retries with doubling delays, completion, no refund | `5 retries (1.1, 2.1, 3.9, 8.1, 18.7 s), settled at +47 s` |
| 4 | Simulator: market closed, then an order | Permanent rejection, on-chain refund, zero retries | `refunded in 3 s, code market_closed` |
| 5 | Pause the workers, place an order, resume | Order waits untouched, completes after resume | `waited 4 min 47 s, settled 7 s after resume` |
| 6 | Reconcile now | Report with no drift | `scanned 9, no drift` |
| 7 | Replay the step 2 event three times | Duplicates ignored, one settlement, broker count unchanged | `3 × duplicate_event_ignored` |

HOW TO READ THE LEDGERS

Each order carries an append-only ledger (`order_events`). A row with an arrow is a status change; a row without one is a fact worth auditing (a retry scheduled, a signature persisted, a duplicate ignored). The row and the status change are written in the same database transaction, so the story survives any crash. Actors: the listener (chain events), a worker (broker and settlement), the reconciler, or a replay.

02 · NORMAL ORDER

# Seven seconds from the chain to the chain.

ORDER 1788788980019 · AAPL · 5.00 USDC
BROKER 02678640-73b1-4066-a163-657631473765 · filled 5.000000
CREATED 4CUnPZ6Z…zirzAsiVm
SETTLED 5VqdsEgg…mEUwiW

| TIME | TRANSITION | REASON | ACTOR |
|---|---|---|---|
| 13:49:41 | recorded | Order seen on the blockchain and recorded event_ingested | listener |
| 13:49:41 | recorded → broker_placing | Intent saved before the broker call broker_intent_recorded | worker |
| 13:49:41 | broker_placing → broker_placed | Broker accepted the trade broker_accepted | worker |
| 13:49:45 | broker_placed → filled | Broker filled the trade broker_filled | worker |
| 13:49:46 | filled → settling | Settling on-chain for the filled amount settlement_started | worker |
| 13:49:46 | settling | Settlement transaction signed and saved before sending fulfill_tx_signed | worker |
| 13:49:47 | settling | Chain emitted BuyOrderFulfilled chain_event_BuyOrderFulfilled | listener |
| 13:49:48 | settling → settled | Settlement confirmed fulfill_confirmed | worker |

Two rows are the crash-safety mechanism: the intent is written before the broker is called, and the settlement signature is persisted before it is sent. A worker that dies after either point resumes from the ledger, never from memory.

03 · BROKER OUTAGE

# Thirty seconds of errors cost latency, not money.

ORDER 1788789130716 · AAPL · 5.00 USDC · placed while the simulator answered HTTP 500
BROKER 81819e9c-779d-4542-af1c-df1947484b7b · filled 5.000000
SETTLED 5P68oHmq…85ey8HM

| TIME | TRANSITION | REASON | ACTOR |
|---|---|---|---|
| 13:52:11 | recorded | Order recorded | listener |
| 13:52:12 | recorded → broker_placing | Intent saved | worker |
| 13:52:12 | broker_placing | Broker 500 · retry 1, wait 1.1 s | worker |
| 13:52:14 | broker_placing | Broker 500 · retry 2, wait 2.1 s | worker |
| 13:52:17 | broker_placing | Broker 500 · retry 3, wait 3.9 s | worker |
| 13:52:21 | broker_placing | Broker 500 · retry 4, wait 8.1 s | worker |
| 13:52:29 | broker_placing | Broker 500 · retry 5, wait 18.7 s | worker |
| 13:52:49 | broker_placing → broker_placed | Outage over, broker accepted | worker |
| 13:52:53 | broker_placed → filled | Broker filled | worker |
| 13:52:58 | settling → settled | Settlement confirmed (signed 13:52:56, chain event 13:52:56) | worker |

A 500 is classified transient: exponential backoff with jitter, at most ten consecutive failures, no refund, and the attempt counter resets once the broker answers. The order was never in danger of being traded twice, because the same deterministic `client_order_id` was sent every time.

04 · MARKET CLOSED

# A permanent refusal is not retried. It is refunded.

ORDER 1788789513875 · AAPL · 5.00 USDC · simulator in market_closed mode
BROKER no broker order exists · rejection code market_closed
REFUNDED 3sL1n55W…RWbXys4er

| TIME | TRANSITION | REASON | ACTOR |
|---|---|---|---|
| 13:58:35 | recorded | Order recorded | listener |
| 13:58:35 | recorded → broker_placing | Intent saved | worker |
| 13:58:35 | broker_placing → refunding | Broker refused for good · MARKET_CLOSED | worker |
| 13:58:36 | refunding | Refund transaction signed and saved before sending | worker |
| 13:58:36 | refunding | Chain emitted BuyOrderRefunded | listener |
| 13:58:38 | refunding → refunded | Refund confirmed | worker |

The difference with step 3 is one classification: a 500 is transient, a market closure is permanent. The permanent path never waits; `refund_buy_order` closes the on-chain ticket and the order ends in its own terminal state, distinct from a dead letter.

05 · KILL SWITCH

# Pause loses nothing. Resume drains everything.

ORDER 1788789621374 · AAPL · 5.00 USDC · placed while the workers were paused
BROKER 0f591089-b06e-4427-8447-9f6378db2661 · filled 5.000000
SETTLED 45NXgGkz…GaKAxu4Zq

| TIME | TRANSITION | REASON | ACTOR |
|---|---|---|---|
| 14:00:22 | recorded | Order recorded · workers paused, nothing else happens for 4 min 47 s | listener |
| 14:05:09 | recorded → broker_placing | Resume · intent saved | worker |
| 14:05:09 | broker_placing → broker_placed | Broker accepted | worker |
| 14:05:14 | broker_placed → filled | Broker filled | worker |
| 14:05:16 | settling → settled | Settlement confirmed (signed 14:05:14, chain event 14:05:15) | worker |

The switch is a row in Postgres read by every worker before it claims work, so all replicas obey it at once. It is the operator's incident tool, and the same switch the circuit breaker throws after a burst of critical alerts.

06 · RECONCILIATION

## Three sources of truth, compared on demand.

`POST /reconcile` at 14:06:24 answered `scanned=9, no drift`. For every terminal order the PendingOrder PDA was closed on-chain; no open order was waiting on a broker state the database did not know. The same job runs unattended every 60 seconds on the hosted stack, and its repairs, when any, land in the ledger with a `reconciled_` reason.

07 · IDEMPOTENT REPLAY

## The same event, three more times. Nothing moves.

ORDER `1788788980019 · the step 2 order, already settled`
REPLAYED `its BuyOrderCreated transaction, decoded from the logs stored in raw_events (origin: db)`

| TIME | STATUS | REASON | ACTOR |
|---|---|---|---|
| 14:26:29 | settled | Same event received again; nothing changed duplicate_event_ignored | replay |
| 14:26:32 | settled | Same event received again; nothing changed | replay |
| 14:26:33 | settled | Same event received again; nothing changed | replay |

After the replays the order still has one `settled` transition and the same settlement signature, and the broker still holds exactly one order for its `client_order_id`. The primary keys of `raw_events` and `orders` make the duplicate harmless before any worker sees it.

NOT COVERED BY THIS MANUAL RUN

Scenario B (worker killed between broker-accept and the database write) and scenario D (settlement confirmation lost, retry hits Anchor error 3012) require killing a process and are run by `pnpm scenario b` and `pnpm scenario d`. Scenario G (real Alpaca sandbox) needs US market hours; its evidence is added to the README when it runs.

VERIFICATION

Every order above can be queried at `GET /orders/BLcWH9MWxKkFNNZj7mBm3vF7aiZieG3BnZMWXtoiktoC/<orderId>/events` on the API, and every signature opens on the devnet explorer.